TP1 : Premiers pas avec git

Mode d'emploi des TPs

Afin de s'adapter au rythme de progression de chacun, les TPs sont pensés pour être suivis de façon autonome. Ils sont construits de la façon suivante :

- Une **partie guidée**, où des concepts sont introduits à travers l'exécution commentée d'une série de commandes, avec un récapitulatif à la fin.
- Des **exercices** pour pratiquer l'utilisation de git.
- Une **antisèche** qui résume toutes les commandes qui ont été introduites.
- Des **informations complémentaires**, qui ne sont pas essentielles mais permettent d'approfondir certains aspects de l'outil.

Les commandes à exécuter dans un terminal sont représentées par du texte en police à chasse fixe, comme ceci :

```
$ whoami
```

Le signe `$` ne fait pas partie de la commande à exécuter et ne doit pas être copié. Il sert à séparer visuellement les commandes en présence de commandes multiples, et il peut être remplacé par d'autres signes donnant des informations supplémentaires. Voici les différents signes que vous allez rencontrer dans les TPs de cette formation :

- `$` désigne des commandes ordinaires que vous pouvez exécuter telles quelles. C'est la grande majorité des commandes du TP.
- `#` désigne des commandes qui ne peuvent être exécutées que par l'administrateur d'un ordinateur. Vous n'êtes pas administrateur de certaines des salles informatiques où la formation peut être jouée, donc ces commandes sont rares et ne sont jamais nécessaires au bon fonctionnement du TP. Elles sont uniquement données pour votre information.
- `?` désigne des commandes qui nécessitent des préparatifs, doivent être modifiées avant exécution, ou plus généralement ont un comportement surprenant. L'énoncé du TP indiquera ce qu'il faut faire.
- `@` désigne des commandes qui vont échouer de façon attendue. La suite du TP vous expliquera pourquoi elles échouent et comment y remédier.

La plupart des commandes git ont un effet contextuel dépendant des autres commandes git qui ont été exécutées auparavant. Par conséquent, la partie guidée du TP suppose que l'ensemble des commandes indiquées ont été exécutées, dans l'ordre indiqué, et à l'exclusion de toute autre commande. Si vous souhaitez faire des expériences pour vérifier que vous avez bien compris les explications, nous vous conseillons de créer un dépôt git à part pour cela afin de ne pas perturber le bon fonctionnement du TP.

L'idée de ce mode de formation est que chacun puisse avancer à son rythme, sans devoir se concentrer sur de longs discours magistraux. Mais le formateur est là pour répondre à toutes vos questions. N'hésitez surtout pas à le solliciter !

Configuration du système

Point de départ

Durant cette formation, vous allez travailler sur des postes de travail Linux équipés des logiciels `git`, `gitg`, `meld` et de quelques éditeurs de texte pour s'adapter aux habitudes de chacun (`nano`, `vim`, `emacs` et `gedit`).

Chaque poste comporte une étiquette avec un nom de machine commençant par « ld-ens » et se terminant par un numéro entre 1 et 20, du style « ld-ens17 ». De ce numéro, vous pouvez déduire le nom du compte utilisateur que vous allez utiliser, dans l'exemple ci-dessus ce serait « formation-git-17 ». Le mot de passe vous sera communiqué oralement.

Ces postes sont aussi configurés pour partager les supports de TP avec le poste enseignant, vous trouverez un raccourci vers ceux-ci sur votre bureau.

Vous l'avez vu, les mots de passe sont dérivés en suivant une logique simple, donc tout le reste de la salle peut deviner votre mot de passe. Si vous souhaitez un peu de vie privée, n'hésitez donc pas à configurer un autre mot de passe un peu plus secret :

```
$ passwd
```

S'identifier auprès de git

Les gestionnaires de versions permettent à plusieurs personnes d'échanger des versions de fichiers entre eux. Il est alors utile de savoir qui a contribué quoi, et comment contacter cette personne.

git vous demandera donc de lui donner votre nom et votre adresse e-mail avant de vous laisser sauvegarder des versions. Voici des exemples des commandes associées, à vous de les adapter pour indiquer votre propre nom et votre propre adresse e-mail !

```
? git config --global user.name 'Francine Dupont'
? git config --global user.email fdupont@serveur-imaginaire.fr
```

Choisir un éditeur de texte

git vous demandera parfois de saisir du texte avec un éditeur de texte. Cet éditeur est configurable, par défaut il s'agit de `vim`.

Le choix d'un éditeur de texte relevant de la guerre de religion, nous dirons juste qu'il est recommandé d'utiliser un éditeur en mode console pour cet usage, et qu'il est conseillé à ceux qui n'ont jamais utilisé ce type d'éditeur de plutôt faire appel à `nano` pour les besoins de cette formation.

Voici comment on configure `nano` comme éditeur par défaut, vous pouvez modifier cette commande pour désigner un des autres éditeurs de texte installés sur le poste de travail si vous le souhaitez :

```
$ git config --global core.editor nano
```

Choisir un nom de branche par défaut

Historiquement, lorsqu'on créait un dépôt git, la branche par défaut s'appelait `master`. Mais ces dernières années ont vu émerger au sein des pays anglo-saxons un mouvement politique contre l'utilisation de la terminologie maître/esclave sous-jacente, qui est perçue comme ayant des connotations post-coloniales malheureuses.

A cause de ça, les versions récentes de git vous encouragent à utiliser un nom de branche par défaut différent. Nous allons ici suivre la convention de GitHub et appeler cette branche `main` :

```
$ git config --global init.defaultBranch main
```

Premiers pas avec la gestion de version

Créer un dépôt

N'importe quel dossier peut être mis sous gestion de version. Il devient alors ce que l'on appelle un « dépôt ». On utilise pour cela la commande `git init` :

```
$ cd ~/Desktop
$ mkdir MonDepot && cd MonDepot
$ git init
```

Comme le message de statut l'indique, cette opération crée un sous-dossier caché « .git », où git stockera ses versions de fichiers.

Statut et ajouts

On peut à tout moment demander quel est l'état actuel du dépôt avec `git status`. Pour l'instant, il ne s'y passe pas grand-chose :

```
$ git status
```

Mais dès la création d'un fichier, le statut change :

```
$ touch MonFichier.txt
$ git status
```

Comme la plupart des gestionnaires de versions, git ne suit pas automatiquement les modifications de tous les fichiers du dossier dans lequel il est actif. Il faut indiquer quels fichiers on souhaite versionner avec `git add` :

```
$ git add MonFichier.txt
$ git status
```

Après cette opération, MonFichier.txt est suivi par git : lorsqu'on enregistrera une version du dépôt (un *commit*), ce fichier y sera présent.

En cas d'ajout non désiré, on peut bien sûr annuler cette opération. Avec une version moderne de git, il suffit d'utiliser `git reset <fichier>`. Sur d'anciennes versions de git, il fallait utiliser pour cet usage la commande plus obscure et spécifique `git rm --cached <fichier>`.

Ignorer des fichiers

Tout contenu ne devrait pas être mis sous gestion de version. Il existe deux raisons classiques d'en exclure des fichiers :

- Ils contiennent des données que l'on ne souhaite pas partager avec autrui (ex : mots de passe, configurations d'outils relevant du goût personnel...)
- Ils peuvent être régénérés à partir des fichiers restants (ex : résultats d'une compilation de code source, journaux d'exécution...)

Créons, pour l'exemple, un fichier `Poubelle.log` que nous ne voudrions pas versionner :

```
$ echo 'DEBUG : Un matin, Gregor Samsa se réveilla' > Poubelle.log
```

Le problème est que `git status` va détecter ce nouveau fichier et nous inciter à le versionner à chaque appel :

```
$ git status
```

Pour éviter ça, on peut créer un fichier `.gitignore` à la racine du dépôt, qui indique que certains fichiers doivent être ignorés par git :

```
$ echo '*.log' > .gitignore
$ git status
```

En général, il est pertinent de mettre `.gitignore` sous gestion de version, son contenu étant sujet à évoluer au fil de l'évolution du projet.

```
$ git add .gitignore
```

`.gitignore` supporte les motifs usuels du shell Unix, comme l'étoile (*wildcard*), et sa configuration s'applique à chaque sous-dossier du dépôt. Pour gérer des situations plus complexes, des syntaxes plus sophistiquées sont disponibles, consultez `git help gitignore` pour plus de détails.

Déplacement et suppression de fichier

À partir du moment où un fichier est sous gestion de version, il est préférable d'informer le gestionnaire de version quand on effectue un déplacement ou une suppression, en utilisant respectivement `git mv` et `git rm`.

```
$ git mv MonFichier.txt MonSuperFichier.txt
$ git status
```

En effet, un gestionnaire de versions n'est pas en position de détecter ces opérations, et si on les effectue avec les outils habituels, elles seront interprétées

par git d'une façon inattendue et indésirable. Prenons l'exemple d'un mouvement de fichier dont git n'a pas été informé :

```
$ mv MonSuperFichier.txt MonIncroyableFichier.txt
$ git status
```

Ici, la commande `mv` standard a ainsi été interprétée comme la suppression d'un fichier sous gestion de version et la création d'un nouveau fichier non versionné. Ce n'est pas ce qui était désiré !

Une commande pratique pour se tirer de ce genre de mauvais pas est `git add --all`, qui intègre à la gestion de version tous les changements d'un répertoire passé en paramètre (ou, si aucun répertoire n'est passé en paramètre, tous les changements du dépôt). Nous pouvons l'utiliser de la façon suivante :

```
$ git add --all .
$ git status
```

Pour ce qui concerne `git rm`, un point à connaître est que git refusera par défaut de supprimer un fichier ayant des changements non enregistrés dans une version, car ces données seraient alors perdues. Ainsi, la commande suivante échouera :

```
@ git rm MonIncroyableFichier.txt
```

Après s'être assuré qu'on souhaite bien perdre ces changements, l'option `--force`, qui s'abrévie en `-f`, peut être utilisée pour confirmer l'opération.

```
$ git rm -f MonIncroyableFichier.txt
```

Gestion des dossiers

Contrairement à votre système d'exploitation, git ne gère qu'une arborescence de *fichiers*. L'existence de dossiers n'est prise en compte par git que dans la mesure où il y a des fichiers à l'intérieur :

```
$ mkdir MonDossier
$ git status
$ echo 'Un semblant de contenu' > MonDossier/Contenu.txt
$ git status
```

Cela ne signifie pas pour autant que git ne supporte pas les dossiers, on peut très bien passer un dossier en paramètre de `git add` par exemple...

```
$ git add MonDossier/
$ git status
```

...mais certaines commandes n'accepteront un dossier en paramètre que si elles sont agrémentées d'une option `--recursive` (qui s'abrévie en `-r`). C'est généralement le cas lorsqu'il est risqué d'effectuer une opération sur un dossier sans connaître son contenu. Par exemple, la commande de suppression suivante échouera.

```
@ git rm MonDossier/
```

Notez que puisque le contenu de ce dossier n'a pas encore été sauvegardé, vous devriez même utiliser `-rf` dans ce cas précis pour convaincre git que vous savez ce que vous faites.

Aide de git

Comme la plupart des outils Unix, git dispose d'une documentation de référence accessible depuis la ligne de commande, soit via la traditionnelle interface `man`, soit via la commande `git help`.

```
$ man git status
$ git help status
```

`git help` reproduit par défaut le comportement de la commande `man`, mais donne aussi accès à quelques documentations généralistes qui ne sont pas accessibles par le biais de `man`, comme `git help everyday`.

En principe, git help donne également accès à des formats de documentation alternatifs, tels que des pages HTML. Mais ces derniers ne sont malheureusement que rarement inclus dans les distributions de git. Si vous souhaitez consulter la documentation au format HTML, il est donc plus sûr de le faire en ligne à l'adresse https://www.git-scm.com/docs.

Si vous ne cherchez pas une explication détaillée du fonctionnement d'une commande, mais seulement un bref rappel de la syntaxe et des options disponibles, vous pouvez aussi utiliser l'option `-h` :

```
$ git mv -h
```

Conclusion

Dans ce premier TP, nous avons vu comment...

- Configurer son système pour l'utilisation de git
- Mettre un répertoire sous gestion de version
- Connaître l'état actuel de git
- Désigner quels fichiers doivent être versionnés ou non
- Effectuer des opérations de base sur ces fichiers
- Utiliser l'aide de git

Mais des zones d'ombres demeurent. Comment créer des versions ? Et quelle est donc cette « branche `main` » dont `git status` ne cesse de parler ? Nous répondrons à ces questions dans les prochaines séquences.

Exercices

1. Créez un dépôt git vide.

2. Configurez le dépôt pour ignorer les fichiers portant l'extension « .o ».
3. Créez un fichier et indiquez à git que vous souhaitez le versionner.
4. Déplacez ce fichier vers un autre emplacement et informez-en git.
   - La sortie de `git status` ne doit parler que de deux fichiers à la fin, celui que vous venez de déplacer et celui qui a été créé à la question 2.
5. Affichez l'état final du dépôt. En ignorant pour l'instant la partie « Sur la branche main », comprenez-vous les autres messages affichés ?

Antisèche

Afficher la configuration actuelle de git :

```
$ git config --list
```

Saisir les identifiants d'utilisateur qui seront insérés dans les commits :

```
? git config --global user.name 'Francine Dupont'
? git config --global user.email fdupont@serveur-imaginaire.fr
```

Définir l'éditeur de texte utilisé par git :

```
? git config --global core.editor nano
```

Initialiser un dépôt git dans le répertoire courant :

```
$ git init
```

Afficher l'état du dépôt git actif :

```
$ git status
```

Mettre un fichier (ou un dossier) sous gestion de version :

```
? git add <fichier>
```

Mettre tous les fichiers du répertoire courant sous gestion de version :

```
$ git add .
```

Intégrer à la gestion de version tous les changements du répertoire courant, y compris les suppressions, les déplacements...

```
$ git add --all .
```

Supprimer un fichier sous gestion de version :

```
? git rm <fichier>
```

Retirer un fichier de la gestion de version sans le supprimer :

```
? git reset <fichier>
```

Déplacer un fichier sous gestion de version :

```
? git mv <source> <destination>
```

Ignorer les fichiers d'extension « .log » (à exécuter à la racine du dépôt) :

```
$ echo '*.log' >> .gitignore
```

Afficher le manuel d'une commande git (ou l'explication d'un concept git) :

```
? git help <commande>
```

Afficher un bref rappel de la syntaxe d'une commande git :

```
? git <commande> -h
```

Informations complémentaires

Interface de configuration

On peut configurer git selon trois périmètres :

1. Ensemble des utilisateurs du système hôte (« system »)
2. Compte utilisateur actif (« global »)
3. Dépôt git actif (« local », choix par défaut)

Quand deux périmètres sont en désaccord, c'est la configuration du périmètre le plus étroit qui est utilisée. Par exemple, si la configuration système spécifie que l'éditeur de texte à utiliser est `vi` et la configuration d'utilisateur spécifie que c'est `emacs`, git utilisera `emacs` comme éditeur de texte.

L'emplacement de la configuration varie d'un système d'exploitation à l'autre. Mais vous n'avez heureusement pas besoin de l'apprendre par cœur, car vous pouvez manipuler cette dernière avec l'utilitaire `git config`, dont voici quelques utilisations fictives :

```
# git config --system reglage 'Valeur pour tout le système'
$ git config --global reglage 'Valeur spécifique à un utilisateur'
$ git config reglage 'Valeur spécifique à un dépôt git'
```

Bien entendu, la manipulation de réglages système nécessite de posséder des droits d'administrateur sur la machine.

La configuration actuelle peut être affichée avec `git config --list`.